

Awantika AIParc 2

March 30, 2026

```
[6]: import heapq

def h(a, b):
    return abs(a[0] - b[0]) + abs(a[1] - b[1])

def astar(grid, start, goal):
    pq = [(0, start)]
    visited = set()
    parent = {}
    g = {start: 0}
    while pq:
        _, curr = heapq.heappop(pq)

        if curr == goal:
            path = []
            while curr in parent:
                path.append(curr)
                curr = parent[curr]
            path.append(start)
            return path[::-1]

        if curr in visited:
            continue

        visited.add(curr)
        x, y = curr

        for dx, dy in [(1,0), (-1,0), (0,1), (0,-1)]:
            nx, ny = x + dx, y + dy

            if 0 <= nx < len(grid) and 0 <= ny < len(grid[0]) and grid[nx][ny] != 0:
                new_g = g[curr] + 1
```

```

        if (nx, ny) not in g or new_g < g[(nx, ny)]:
            g[(nx, ny)] = new_g
            f = new_g + h((nx, ny), goal)

            heapq.heappush(pq, (f, (nx, ny)))
            parent[(nx, ny)] = curr

    return None

# Example Input

grid = [
    [0, 0, 0],
    [1, 0, 1],
    [0, 0, 0]
]

start = (0, 0)
goal = (2, 2)

path = astar(grid, start, goal)
print("Shortest Path:", path)

```

Shortest Path: [(0, 0), (0, 1), (1, 1), (2, 1), (2, 2)]